

Software Engineer Case

Software Engineering Case

Background

At Spoor, engineering work often sits between real-world data, uncertain model output, operational constraints, and product needs. This case is a small version of that kind of problem.

Spoor builds technology for monitoring bird activity around wind farms. Detecting and understanding birds in video is therefore close to our real product domain, even though this case uses a simplified video and a small prototype scope.

You will receive a short video containing birds and other objects in a scene. Your task is to build a small system that processes the video and produces useful output about what appears in it over time.

This is not a computer vision accuracy contest. We do not expect a production-ready system in a few hours. We are interested in how you think through an open-ended problem, how you structure your solution, and how clearly you communicate the choices you made.

Expected time: **3-5 hours**.

The Task

Build a solution that:

- Processes the provided video.
- Detects and tracks objects over time.
- Produces a structured output file with the results.
- Provides some way to inspect or understand the output.
- Includes enough validation that you can explain why you trust the result.

You may use any programming language, framework, library, pre-trained model, or AI-assisted coding tool you prefer.

Object Detection And Tracking

Develop an object detector and object tracker that can identify and follow objects across frames in the provided video.

You should leverage any suitable techniques, frameworks, or pre-trained models. For example, Python, PyTorch, TensorFlow, OpenCV, YOLO, SORT, DeepSORT, or similar tools are all acceptable.

Your solution should make a reasonable attempt to:

- Detect birds and other relevant objects in the scene.
- Assign each tracked object a persistent `track_id` across frames.
- Classify detections as `bird` or `non-bird`, if your chosen approach supports it.

You will not be evaluated on achieving high detection, tracking, or classification accuracy. Use the approach you think is most appropriate for the time available, and explain the limitations.

Output

Your system should produce a machine-readable result file in a standard format such as CSV, JSONL, SQLite, Parquet, or similar.

The output should contain detection and tracking information frame by frame.

Include at least the following fields:

Field	Description
<code>frame_id</code>	Sequential frame number.
<code>bbox</code>	Bounding box coordinates. Use a clear format, for example normalized <code>(x, y, h, w)</code> or pixel coordinates. Document which one you use.
<code>track_id</code>	Persistent identifier for each tracked object.
<code>class</code>	Predicted class label, for example <code>bird</code> or <code>non-bird</code> .
<code>timestamp</code>	Corresponding video time.

You may include additional fields, such as confidence scores, model labels, or processing metadata, if useful.

Inspection

Provide a way to inspect the result. This can be simple.

For example:

- An annotated video with bounding boxes, track IDs, and class labels overlaid.
- Frame-by-frame image outputs with bounding boxes and class predictions.
- A script, or small local UI for reviewing detections.

- Any additional visualization that improves clarity of results.

Validation

Include tests, checks, or other validation that you consider useful for this prototype.

We are not asking for exhaustive coverage. We want to see what you choose to validate, why it matters, and how you handle uncertainty or edge cases.

Writeup

Include a short writeup with your submission. Prefer clarity over length.

Please cover:

- How you thought about the problem.
- The main components of your solution.
- Important tradeoffs and assumptions.
- What you intentionally left out.
- What you would improve with more time.
- How you would think about running something like this outside your local machine, for example near cameras or on edge hardware.
- How you used AI or automation tools, if applicable.

For AI/tool usage, include enough detail for us to understand how the tools shaped your work. We are interested in what you asked for help with, what you accepted or changed, and where you made the key decisions yourself.

A full AI chat log or agent history is preferred where possible. If you cannot provide the full log, include the most relevant excerpts and summarize what happened in the parts you omit.

Submission

Please send:

- Source code.
- Run instructions.
- Test or validation instructions.
- Your generated output file.
- Any generated inspection artifacts, unless they are produced by running the project.
- The short writeup.
- AI chat log or agent history, where possible.

Package the submission in a way that makes it straightforward to run locally. A container is welcome but not required.